SUPWARDEN

Documentation Technique

Gestionnaire de mots de passe collaboratif

Projet Fullstack Web - PHP / Symfony 6.4 + React 19

Sommaire

Sommaire 2
1. Introduction 6
2. Contexte et objectifs 6
2.1 Problématique 6
2.2 Objectifs du projet 6
2.3 Positionnement 6
3. Périmètre fonctionnel réalisé 6
4. Architecture générale 7
4.1 Vue d'ensemble 7
4.2 Diagrammes d'architecture 8
4.3 Diagramme des cas d’utilisation 8
4.4 Diagramme de classes 9
4.5 Diagramme de séquence d'authentification 10
4.6 Diagramme de séquence de trousseaux 11
4.7 Diagramme de séquence des éléments (Secrets) 12
4.8 Diagramme de séquence des messages 13
5. Stack technique et justification des choix 13
5.1 Tableau de la stack 13
5.2 Justification des choix clés 14
Symfony 6.4 LTS 14
Jeton Bearer personnalisé (sans bibliothèque JWT) 14
Chiffrement libsodium 15
React 19 sans bibliothèque de routing tierce 15
PostgreSQL 16 15
Mercure Hub 15
6. Structure du projet 16
6.1 Commandes Symfony CLI 17
7. Modèle de données 18
7.1 Schéma Entité-Relation (ERD) 18
7.2 Description détaillée des tables 18
sw_app_user 18
sw_vault 19

sw_vault_member 19
sw_vault_item 19
sw_item_permission 19
sw_attachment 19
sw_private_message et sw_vault_message 19
7.3 Indexes notables 19
8. Règles de sécurité 20
8.1 Authentification 20
Jeton Bearer 20
Hachage des mots de passe 20
Code PIN 20
8.2 Autorisation 20
Pare-feu stateless 20
Voters Symfony 21
8.3 Chiffrement des secrets 21
8.4 Limitation de débit (Rate Limiting) 21
8.5 CORS 22
8.6 Invalidation de session 22
9. API et contrats d'échange 22
9.1 Conventions générales 22
Format d'erreur standard 22
Tracabilité 22
9.2 Tableau des routes 23
Authentification et profil 23
Trousseaux 23
Éléments 23
Messagerie 24
Outils, import/export et statistiques 24
Administration et système 24
9.3 Exemples de contrats d'échange 25
POST /api/auth/register 25
POST /api/vaults/{id}/items 25
10. Fonctionnalités majeures implémentées 26
10.1 Trousseaux et collaboration 26

10.2 Déverrouillage des secrets sensibles 26
10.3 OAuth Google (PKCE) 26
10.4 Messagerie temps réel 26
10.5 Import / Export 27
11. Tests et stratégie de validation 27
11.1 Stratégie globale 27
11.2 Tests backend (PHPUnit) 27
11.3 Tests frontend (Vitest + Playwright + Cypress) 28
12. Déploiement et exécution 28
12.1 Prérequis 28
12.2 Configuration initiale 28
12.3 Variables d'environnement critiques 29
12.4 Démarrage en développement 29
12.5 Démarrage en production 29
12.6 Santé des services 29
12.7 Commandes de maintenance 30
13. Prise en main développeur 30
13.1 Prérequis 30
13.2 Installation pas à pas 30
13.3 Variables d'environnement détaillées 31
Variables obligatoires pour démarrer 31
Variables OAuth Google (optionnelles en développement) 32
13.4 Services et ports 32
13.5 Cartographie du code 32
Backend (apps/api/src/) 32
Frontend (apps/web/src/) 33
13.6 Flux de développement quotidien 33
13.7 Migrations de base de données 33
13.8 Lancer les tests 34
13.9 Reconstruire les images Docker 34
13.10 Problèmes fréquents 34
14. Difficultés rencontrées et solutions 35
14.1 Invalidation de jetons sans état (stateless) 35
14.2 Chiffrement côté serveur et portabilité 35

14.3 Calcul dynamique du type de trousseau 35
14.4 Routage SPA frontend sans React Router 35
14.5 Environnement de test unifié 36
15. Limites connues 36
16. Perspectives d'amélioration 36
16.1 Sécurité 36
16.2 Fonctionnel 36
16.3 Infrastructure 37
17. Annexes techniques 37
17.1 Configuration Docker Compose complète (développement) 37
17.2 Structure du jeton Bearer 37
17.3 Format de stockage des secrets chiffrés 38
17.4 Entités Doctrine - champs techniques 38
Champ roles (sw_app_user) 38
Triggers Doctrine (#[ORM\PrePersist] / #[ORM\PreUpdate]) 38
17.5 Documentation OpenAPI 38
17.6 Commandes utiles 38

Date : Avril 2026

Projet : SUPWARDEN - Gestionnaire de mots de passe collaboratif

Nature : Document technique à destination des examinateurs

1. Introduction

Ce document constitue la documentation technique du projet **SUPWARDEN**, un gestionnaire de mots de passe collaboratif développé dans le cadre d'un projet fullstack web. Il s'adresse à un lectorat technique et a pour vocation de décrire précisément l'architecture, les choix d'implémentation, le modèle de données, les contrats d'API et la stratégie de déploiement du système.

Le document se concentre exclusivement sur les aspects techniques et architecturaux de la solution.

Toutes les affirmations formulées dans ce document sont basées sur le code source réellement implémenté dans le dépôt. Aucune fonctionnalité non présente dans le code source n'est décrite.

2. Contexte et objectifs

2.1 Problématique

La gestion sécurisée des secrets numériques (mots de passe, clés, identifiants) constitue un enjeu central dans les contextes individuels et collaboratifs. Les solutions existantes sont soit propriétaires et opaques, soit insuffisamment adaptées au travail en équipe avec des niveaux d'accès différenciés.

2.2 Objectifs du projet

SUPWARDEN vise à fournir :

- Un stockage chiffré de secrets numériques organisés en **trousseaux** (*vaults*)
- Un modèle de **collaboration à trois niveaux de rôle** (propriétaire, éditeur, lecteur)
- Des **permissions fines par élément** permettant de contrôler lecture, édition, pièces jointes et révélation de secret
- Une **communication intégrée** par messagerie privée et par trousseau
- Un accès sécurisé depuis n'importe quel navigateur web, sans installation cliente
- Une infrastructure **déployable par conteneurs** Docker

2.3 Positionnement

SUPWARDEN se distingue des gestionnaires de mots de passe grand public par son architecture ouverte, son déploiement auto-hébergé et son modèle de partage granulaire. Il ne prétend pas offrir le niveau de sécurité d'un coffre-fort certifié ; ses mécanismes de protection sont solides et documentés, mais le chiffrement est côté serveur et non bout-en-bout.

3. Périmètre fonctionnel réalisé

Le tableau ci-dessous liste l'ensemble des fonctionnalités effectivement implémentées dans le code source à la date de ce document.

Domaine	Fonctionnalité	Statut
Authentification	Inscription avec email/mot de passe	Oui
	Connexion et émission d'un jeton Bearer	Oui
	Déconnexion avec invalidation du jeton	Oui
	OAuth 2.0 Google (PKCE)	Oui
	Code PIN (4-6 chiffres) pour les secrets sensibles	Oui
	Limitation de taux sur la connexion (429 après 5 échecs)	Oui
Profil	Modification du profil (nom, mot de passe)	Oui
	Suppression de compte	Oui
Trousseaux	Création, lecture, mise à jour, suppression	Oui
	Type PERSONAL/SHARED calculé dynamiquement	Oui
	Recherche par nom, identifiant ou url	Oui
Membres	Invitation par email, attribution d'un rôle	Oui
	Modification de rôle par le propriétaire	Oui
	Quitter un trousseau (sauf propriétaire)	Oui
Éléments	CRUD complet (nom, identifiant, secret, notes)	Oui
	URIs et pieces jointes multiples par élément	Oui
	Champs personnalisés (texte, sensible)	Oui
	Chiffrement des secrets (libsodium XSalsa20-Poly1305)	Oui
	Masquage du secret (hasSecret, déverrouillage explicite)	Oui
	Marquage sensible (révélation via PIN ou mot de passe)	Oui
Permissions	Permissions fines par élément	Oui
Pièces jointes	Upload, téléchargement, prévisualisation, suppression	Oui
Messagerie	Messagerie privée instantané entre utilisateurs	Oui
	Chat de groupe instantané par trousseau	Oui
Temps réel	Notifications Mercure (SSE)	Oui
Outils	Générateur de mots de passe	Oui
	Import et Export JSON/CSV des trousseaux	Oui
	Export JSON/CSV d'un trousseau	Oui
Statistiques	Tableau de bord chiffré (vaults, items, membres)	Oui
API	Documentation OpenAPI auto-générée (/api/doc)	Oui

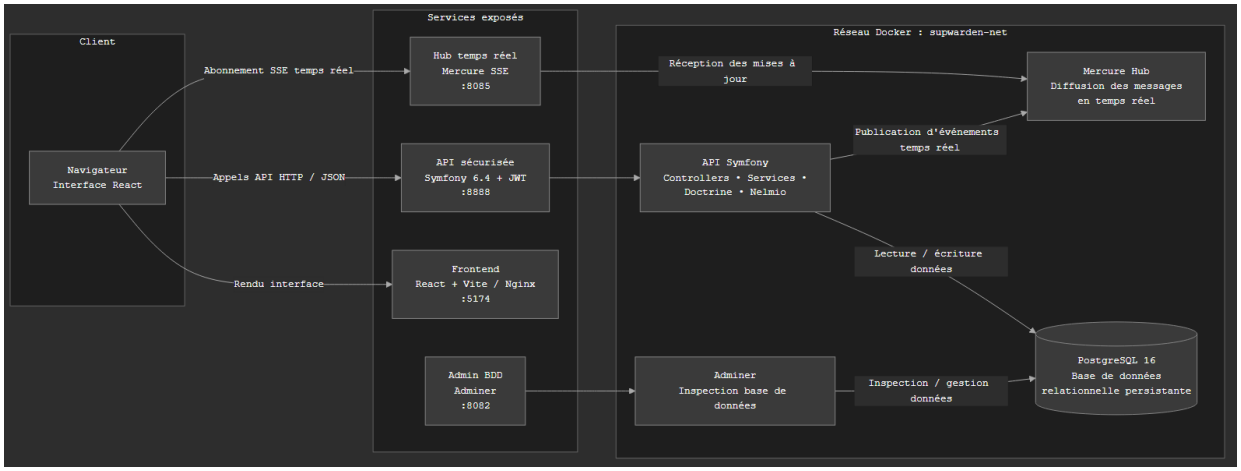
4. Architecture générale

4.1 Vue d'ensemble

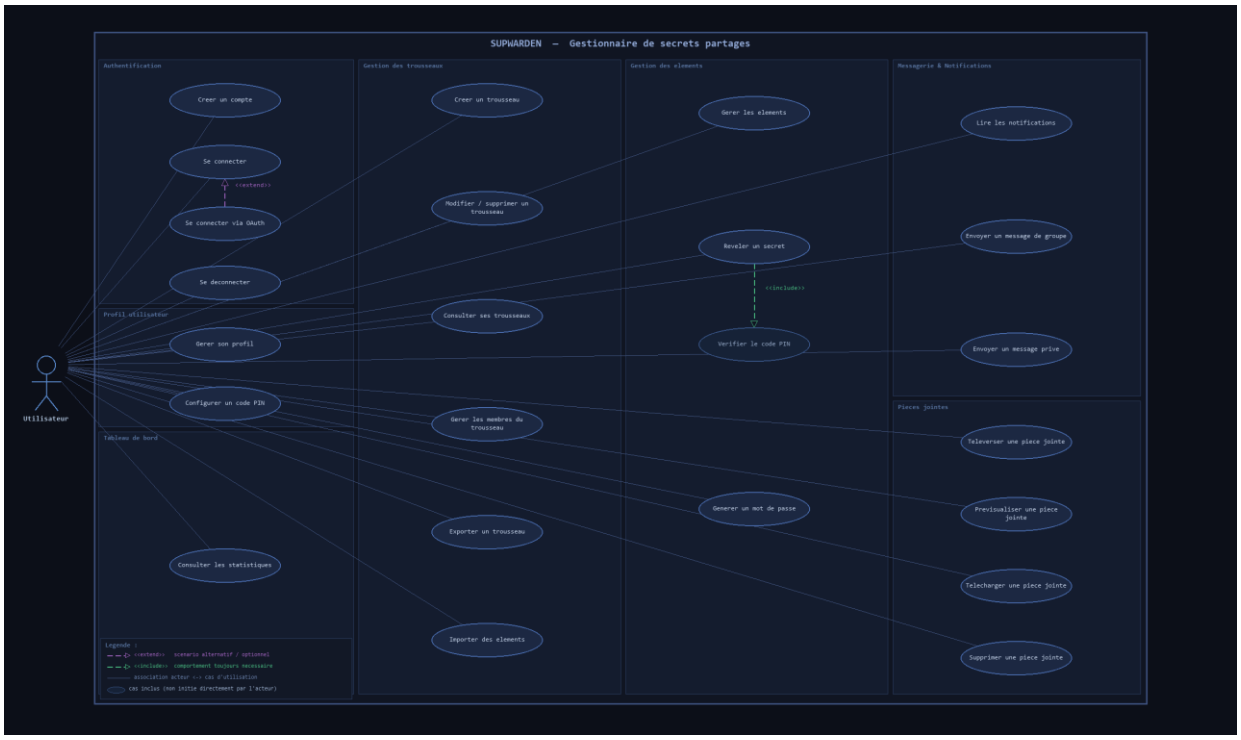
SUPWARDEN est une application **monorepo** organisée en deux sous-projets indépendants déployés ensemble par Docker Compose. Le frontend communique exclusivement avec le backend via une API REST. Le backend émet des événements temps réel via un serveur Mercure distinct.

```
+-----------------------------------------------------------------+
|                        Navigateur Web                           |
|            React 19 SPA  (http://localhost:5173)                |
+-------------------+-------------------------+-------------------+
                    | REST/JSON (Bearer token)  | SSE (Mercure)
                    v                           v
+----------------------------+   +-------------------------------+
|  Backend -- Symfony 6.4    |   |   Mercure Hub (Caddy/Go)      |
|  PHP 8.1+  (:8000)         |-->|   (:3000)                     |
|  API REST + OpenAPI        |   |   Pub/Sub Server-Sent Events  |
+--------------+-------------+   +-------------------------------+
               | Doctrine ORM
               v
+----------------------------+
|  PostgreSQL 16 (:5432)     |
|  11 tables applicatives    |
+----------------------------+
```

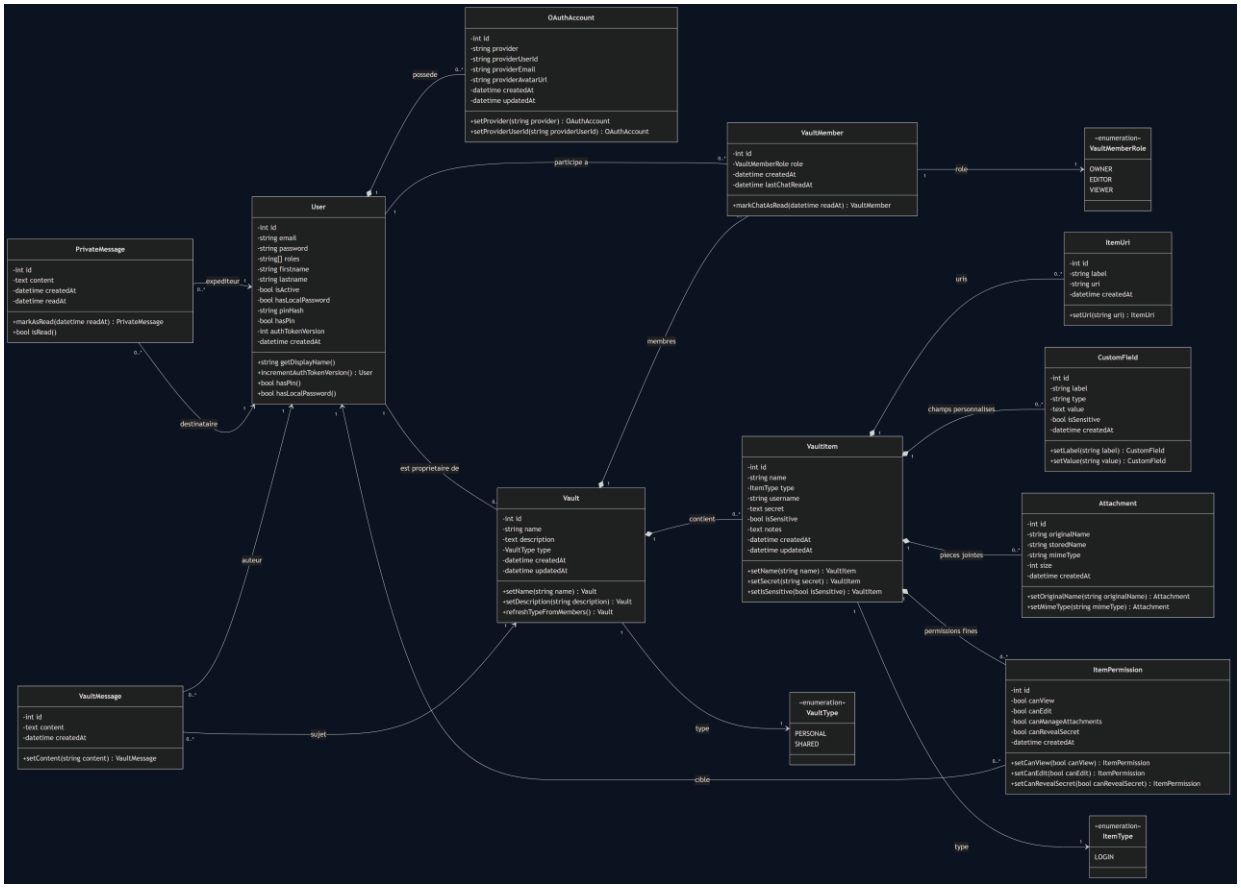
4.2 Diagrammes d'architecture



4.3 Diagramme des cas d'utilisation

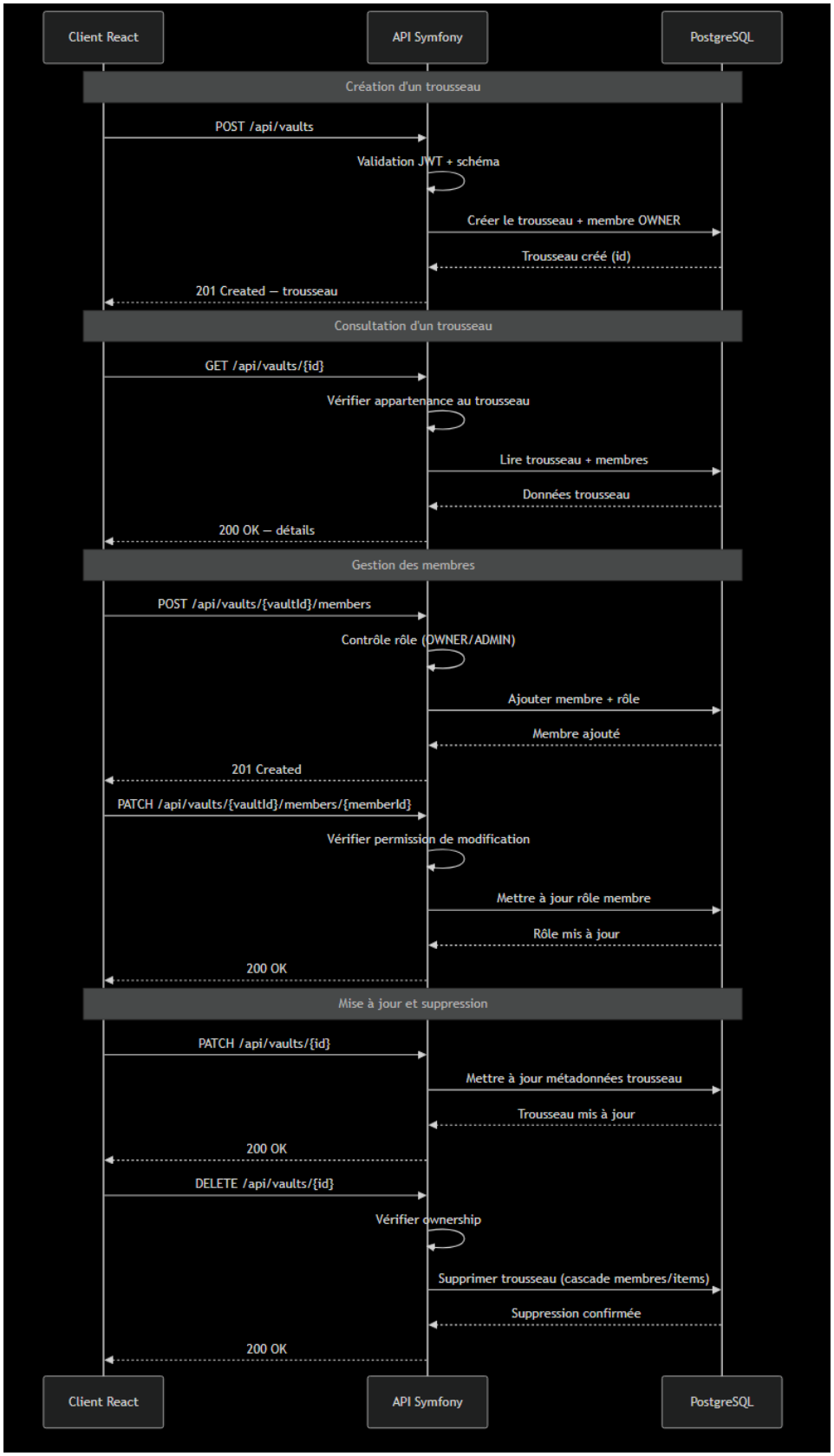


4.4 Diagramme de classes

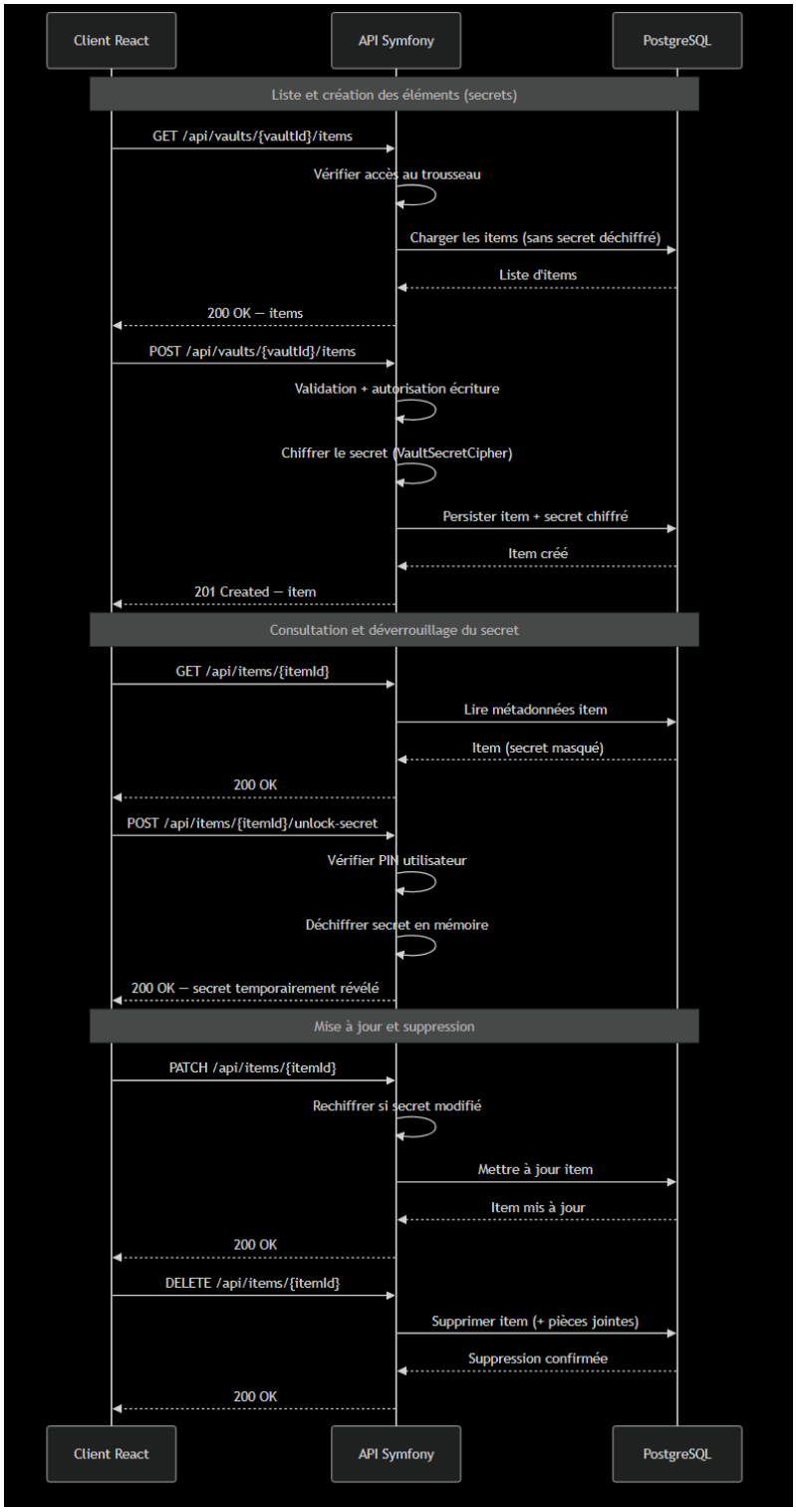


4.5 Diagramme de séquence d'authentification

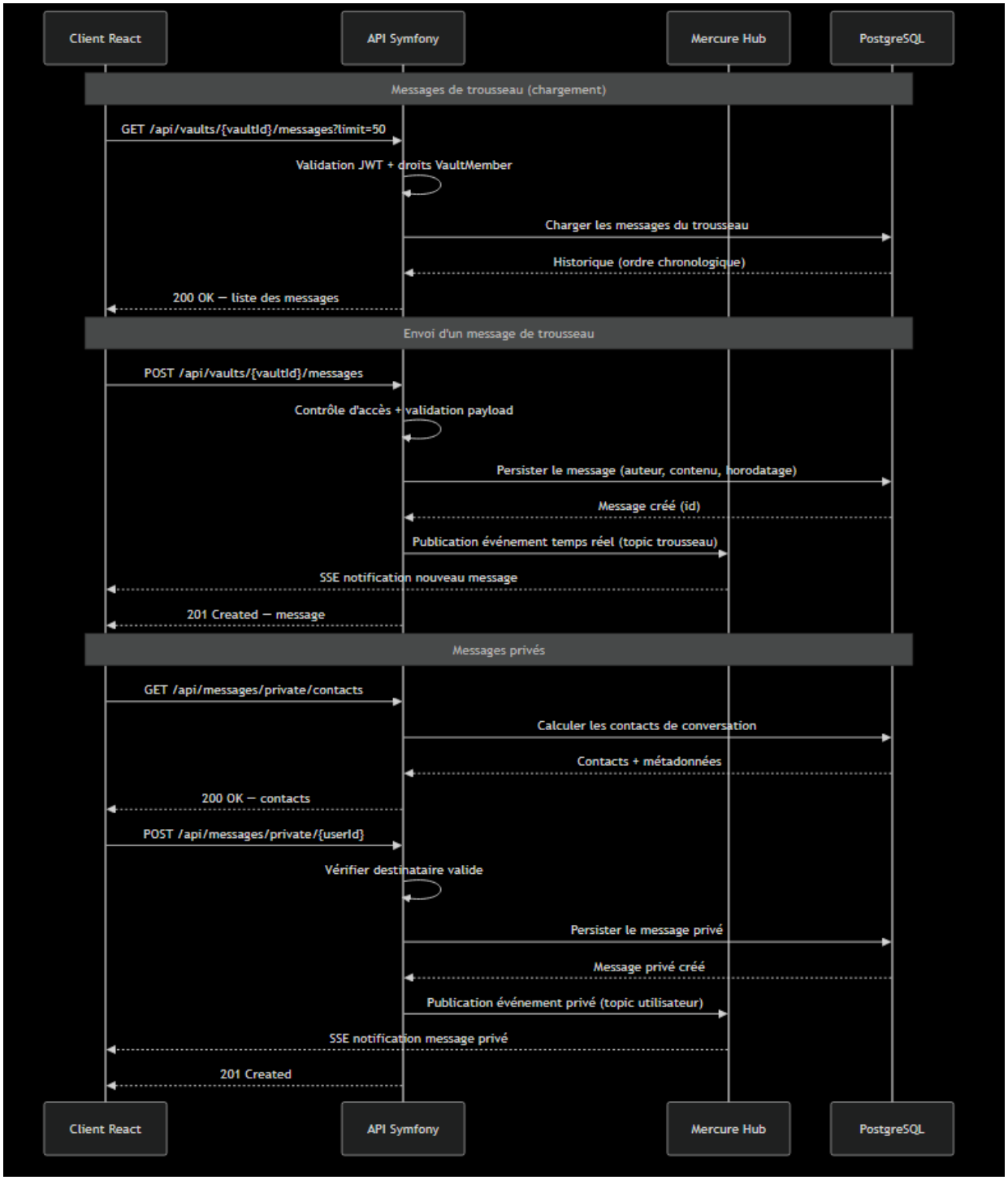
4.6 Diagramme de séquence de trousseaux



4.7 Diagramme de séquence des éléments (Secrets)



4.8 Diagramme de séquence des messages



5. Stack technique et justification des choix

5.1 Tableau de la stack

Composant	Technologie	Version	Rôle

Backend	Symfony	6.4.* (LTS)	Framework PHP, routage, DI, validation
Langage backend	PHP	>= 8.1	Typage strict, attributs natifs, fibres
ORM	Doctrine ORM	^3.6	Abstraction BDD, migrations, entités
Base de données	PostgreSQL	16-alpine	Stockage relationnel de production
Sécurité API	symfony/security-bundle	6.4.*	Authentificateurs, voters, access control
Chiffrement	libsodium (ext-sodium)	natif PHP 7.2+	XSalsa20-Poly1305 pour les secrets
Limitation de taux	symfony/rate-limiter	6.4	Brute-force protection sur /auth/login
Documentation API	nelmio/api-doc-bundle	^5.7	Génération OpenAPI 3.0 automatique
CORS	nelmio/cors-bundle	^2.6	Autorisation cross-origin frontend <-> backend
Temps réel	Mercure Hub (Caddy)	latest	Server-Sent Events, pub/sub
Frontend	React	^19.2.4	SPA, composants, contextes
Build frontend	Vite	^5.4.19	Build rapide, HMR, mode dev Docker
Tests backend	PHPUnit	^9.5	Tests fonctionnels d'intégration
Tests frontend unit.	Vitest	^2.1.8	Tests unitaires composants (jsdom)
Tests E2E	Cypress	^13.17.0	Tests E2E avec helpers personnalisés
Conteneurisation	Docker + Compose	v2	Orchestration des services locaux
Interface BDD	Adminer	4	Console SQL pour développement

5.2 Justification des choix clés

Symfony 6.4 LTS

Symfony 6.4 est la version en support long terme (maintenance jusqu'en 2027). Sa maturité garantit une base stable pour un projet académique. Le système de voters, le composant Security et le Rate Limiter ont été exploités directement sans sur-couche additionnelle.

Jeton Bearer personnalisé (sans bibliothèque JWT)

Le projet s'appuie sur un gestionnaire de jetons Bearer développé sur mesure, en remplacement de LexikJWTAuthenticationBundle. Le jeton utilise le format **Base64Url(payload).Base64Url(HMAC-**

SHA256(payload, APP_SECRET)), garantissant l'intégrité des données transmises tout en conservant une structure légère. Cette implémentation permet de limiter les dépendances externes, de maîtriser entièrement le contenu du payload et d'adapter le fonctionnement de l'authentification aux besoins spécifiques de l'application. Le champ version, intégré au payload, permet par ailleurs d'invalider l'ensemble des jetons actifs d'un utilisateur, notamment lors d'un changement de mot de passe ou d'une déconnexion globale.

Chiffrement libsodium

Le projet utilise l'algorithme **XSalsa20-Poly1305** fourni par **l'extension native Sodium** disponible en PHP depuis la version 7.2. Cette solution assure simultanément la confidentialité et l'authentification des données au sein d'un même mécanisme cryptographique, réduisant les risques d'erreurs liés à l'assemblage de plusieurs algorithmes distincts. La clé de chiffrement est dérivée à partir de **APP_SECRET** via **hash('sha256', APP_SECRET, true)**. Les données chiffrées sont enregistrées en base selon le format **enc:v1:<base64(nonce||ciphertext)>**, où le préfixe **enc:v1:** permet d'anticiper d'éventuelles évolutions du système de chiffrement tout en conservant la compatibilité avec les données existantes.

React 19 sans bibliothèque de routing tierce

Le frontend repose sur React 19 avec un routeur interne implémenté dans **AppRouter.jsx**, basé sur **window.history.pushState()** et un matching explicite des chemins. Cette approche supprime la dépendance à une librairie de routing externe tout en conservant une maîtrise fine des transitions, des gardes d'accès et des redirections applicatives.

PostgreSQL 16

Choix de production standard, image Alpine légère, support des types JSONB si besoin futur, contraintes **UNIQUE** et indexes composites utilisés explicitement dans les entités.

Mercure Hub

Server-Sent Events plutôt que WebSocket pour les notifications temps réel. SSE est unidirectionnel (serveur -> client) ce qui correspond aux besoins (notifications de nouveaux messages, événements vault). Mercure est un hub Caddy open source avec support JWT natif pour la sécurité des canaux.

6. Structure du projet

```
supwarden/
├── .docker-compose.yml  # Orchestration Docker (développement)
├── .docker-compose.prod.yml  # Orchestration Docker (production)
├── .env.example  # Variables d'environnement — template dev
├── .env.prod.example  # Variables d'environnement — template prod
├── .env.local.example  # Surcharge locale (non versionnée)
├── infra/
│   └── docker/
│       ├── api/Dockerfile  # Image PHP 8.1-cli-alpine
│       ├── web/Dockerfile  # Image Node 20 + Nginx
│       ├── db/init.sql  # Script d'initialisation PostgreSQL
│       └── mercure/Caddyfile  # Configuration du hub Mercure
├── scripts/
│   └── start-api.sh  # Démarrage API — migrations + serve
├── docs/
│   ├── user-guide.md  # Manuel utilisateur
│   └── technical-documentation.md  # Ce document
└── apps/
    ├── api/  # Backend Symfony 6.4
    │   ├── composer.json
    │   ├── config/
    │   │   ├── packages/
    │   │   │   ├── security.yaml
    │   │   │   ├── doctrine.yaml
    │   │   │   └── nelmio_cors.yaml
    │   │   └── routes/
    │   ├── migrations/  # 17+ fichiers de migration Doctrine
    │   ├── src/
    │   │   ├── Controller/  # 17 contrôleurs organisés par domaine
    │   │   │   ├── Auth/  # AuthController, GoogleOAuthController
    │   │   │   ├── User/  # MeController, ProfileController
    │   │   │   ├── Vault/  # VaultController, VaultMemberController, VaultExportController
    │   │   │   ├── Item/  # ItemController, ItemAttachmentController
    │   │   │   ├── Messaging/  # MessagingController
    │   │   │   ├── Notification/  # NotificationController
    │   │   │   ├── Tools/  # PasswordGeneratorController, ImportExportController, StatsController
    │   │   │   ├── Admin/  # UserAdminController
    │   │   │   ├── System/  # HealthController
    │   │   │   └── Documentation/  # ApiDocumentationController
    │   │   ├── Entity/  # 11 entités Doctrine (préfixe sw_)
    │   │   ├── Command/  # 3 commandes Symfony CLI
    │   │   ├── Repository/  # 11 repositories
    │   │   ├── Dto/  # Objets d'entrée validés (RegisterInput, LoginInput...)
    │   │   ├── Enum/  # Constantes métier (VaultMemberRole, VaultType, ItemType)
    │   │   ├── EventSubscriber/  # Hooks transversaux (ApiRequestLifecycleSubscriber)
    │   │   ├── Security/
    │   │   │   ├── Auth/  # ApiTokenAuthenticator, AuthRateLimiter
    │   │   │   ├── Token/  # BearerTokenManager
    │   │   │   ├── Vault/  # VaultVoter
    │   │   │   └── Item/  # ItemVoter
    │   │   └── Service/  # Logique métier (cipher, export, import...)
    │   └── tests/
    │       └── Functional/  # 5 suites de tests + ApiTestCase
    └── web/  # Frontend React 19
        ├── package.json
        ├── vite.config.js
        ├── src/
        │   ├── main.jsx
        │   ├── App.jsx
        │   ├── context/  # AuthContext.jsx — état global d'auth
        │   ├── routes/  # AppRouter.jsx — routing SPA personnalisé
        │   ├── pages/  # Pages organisées par domaine
        │   │   ├── auth/  # Login, Register, OAuthCallback
        │   │   ├── dashboard/
        │   │   ├── vaults/  # VaultList, VaultDetail, VaultCreate
        │   │   ├── items/  # ItemDetail, ItemForm
        │   │   ├── messages/  # MessagesPage
        │   │   ├── settings/  # ProfilePage
        │   │   └── marketing/  # LandingPage, BrandPage
        │   ├── services/api/  # 5 modules de service API
        │   │   ├── authApi.js
        │   │   ├── vaultApi.js
        │   │   ├── itemApi.js
        │   │   ├── messagingApi.js
        │   │   └── advancedApi.js
        │   ├── components/  # Composants réutilisables
        │   ├── hooks/  # Hooks React personnalisés
        │   ├── layouts/  # Structures de page partagées
        │   ├── utils/  # Fonctions utilitaires
        │   └── styles/  # Variables et styles globaux
        ├── playwright/  # Tests E2E Playwright
        └── cypress/  # Tests E2E Cypress
```

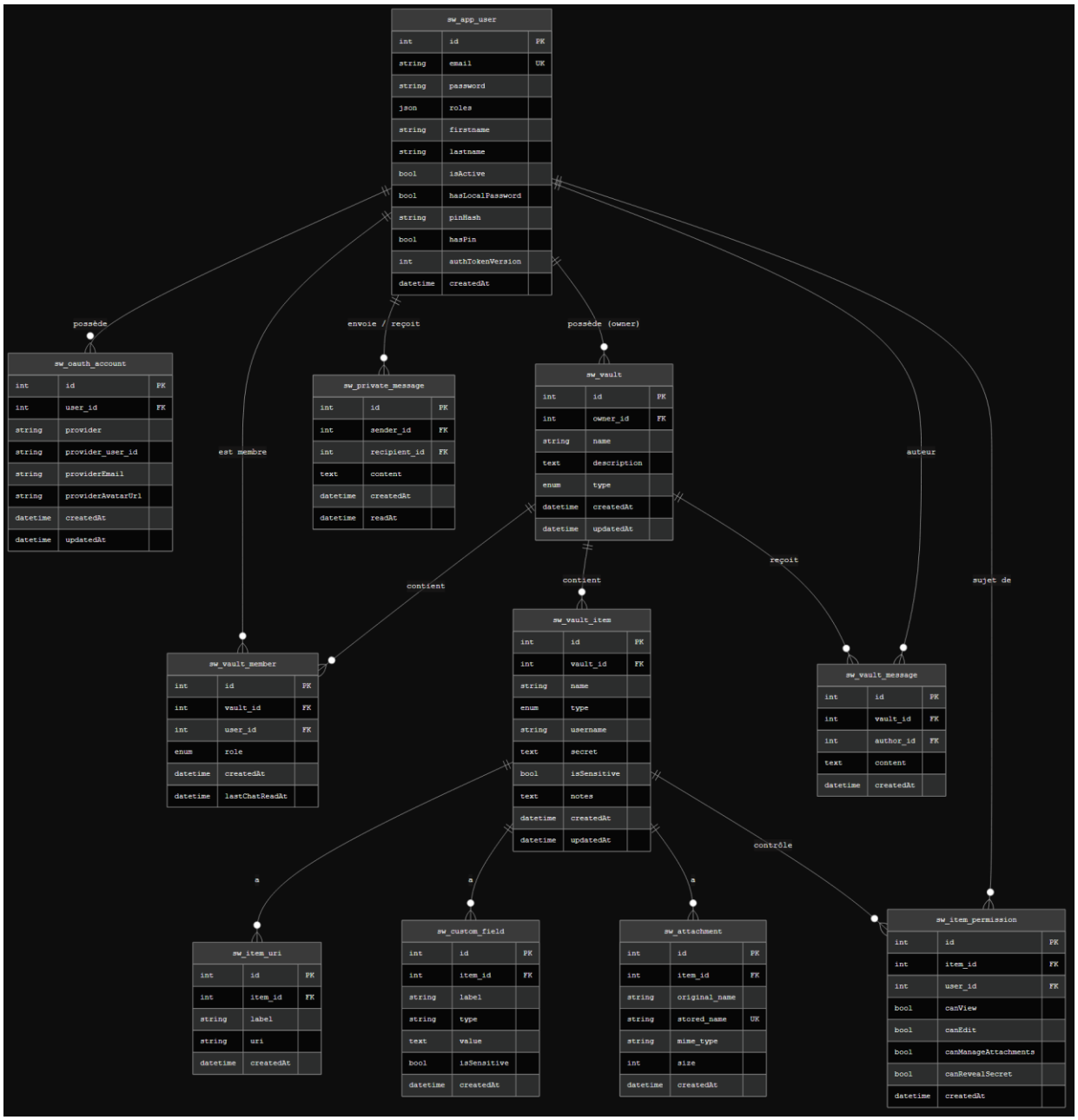
6.1 Commandes Symfony CLI

Trois commandes sont disponibles dans apps/api/src/Command/ :

Commande	Description
EncryptLegacyItemSecretsCommand	C'est une commande de migration/durcissement des données existantes. Elle parcourt les items dont le secret est encore en clair (non préfixé enc:v1:), les chiffre avec VaultSecretCipher, puis enregistre le résultat en base.
SeedUserVaultsCommand	Pour tests manuels, captures UI et démos. Elle génère des données de démonstration pour un utilisateur cible: création de plusieurs trousseaux et d'items associés, ou ajout d'items dans un trousseau existant (--target-vault). Utile pour tests manuels, captures UI et démos.
EnsurePersonalVaultsCommand	C'est une commande de cohérence métier. Elle vérifie que chaque utilisateur possède un trousseau personnel système valide (owner correct, membership owner présent) et corrige automatiquement les écarts détectés.

7. Modèle de données

7.1 Schéma Entité-Relation (ERD)



7.2 Description détaillée des tables

sw_app_user

Table centrale représentant les comptes utilisateurs. Le champ **authTokenVersion** (entier, incrémenté à chaque changement de mot de passe ou déconnexion) permet l'invalidation immédiate de tous les jetons actifs d'un utilisateur. Le champ **hasLocalPassword** distingue les comptes créés par email/mot de passe des comptes OAuth purs.

sw_vault

Un trousseau appartient à un propriétaire (owner_id). Le champ type (enum PERSONAL/SHARED) est calculé dynamiquement par l'API en fonction du nombre de membres distincts du propriétaire ; il n'est pas stocké dans la base mais renvoyé dans les réponses.

Note de conception : une contrainte de suppression en cascade (CASCADE) s'applique sur owner_id. La suppression d'un utilisateur entraîne la suppression de ses trousseaux possédés.

sw_vault_member

Table de jonction enrichie entre sw_vault et sw_app_user. La contrainte UNIQUE(vault_id, user_id) empêche les doublons. Le champ lastChatReadAt permet de calculer le nombre de messages non lus par trousseau.

Roles : OWNER, EDITOR, VIEWER

sw_vault_item

Le champ secret contient le secret chiffré au format **enc:v1:<base64(nonce||ciphertext)>.** Dans les réponses de liste (**GET /api/vaults/{id}/items**), le secret n'est jamais transmis ; seul le booléen hasSecret indique sa présence. La révélation nécessite un appel explicite à **POST /api/items/{id}/unlock-secret**.

sw_item_permission

Table de permissions fines par élément et par utilisateur. Les 4 états disponibles sont :

Etats	Description
canView	Voir l'élément dans la liste
canEdit	Modifier l'élément
canManageAttachments	Uploader / supprimer des pièces jointes
canRevealSecret	Déverrouiller et lire le secret

sw_attachment

Les fichiers sont stockés sous un nom généré (stored_name, unique). Le nom original (original_name) est conservé pour le téléchargement. La contrainte UNIQUE(stored_name) évite les collisions.

sw_private_message et sw_vault_message

Deux tables distinctes pour les deux types de messagerie. **readAt** sur **sw_private_message** permet de calculer les messages non lus par conversation.

7.3 Indexes notables

Table	Index	Colonnes	Justification
sw_vault	idx_owner	owner_id	Lister les trousseaux d'un utilisateur
sw_vault_member	idx_vault_user (UNIQUE)	(vault_id, user_id)	Contrainte d'unicité + lookup
sw_vault_item	idx_vault_created	(vault_id, created_at)	Listage paginé d'éléments
sw_item_permission	idx_item_user (UNIQUE)	(item_id, user_id)	Une permission par couple

sw_private_message	idx_conversation	(sender_id, recipient_id, created_at)	Reconstruction d'une conversation
sw_vault_message	idx_vault_created	(vault_id, created_at)	Historique de chat par trousseau

8. Règles de sécurité

8.1 Authentification

Jeton Bearer

Le BearerTokenManager produit un jeton signé au format :

```
<Base64Url(JSON_payload)>.<Base64Url(HMAC-SHA256(payload, APP_SECRET))>
```

Payload :

```
{
  "sub": 42,
  "iat": 1713600000,
  "exp": 1713686400,
  "version": 3
}
```

La validation (parse()) effectue dans l'ordre :

1. Vérification du format (exactement 2 parties séparées par .)
2. Recalcul de la signature et comparaison par hash_equals() (protection contre les attaques temporelles)
3. Vérification de l'expiration (exp < time())
4. Vérification que version correspond à user.authTokenVersion en base

Durée de vie : configurable via TOKEN_TTL (défaut 24 heures).

Hachage des mots de passe

Configuration Symfony (security.yaml) : password_hasher: 'auto'. En production, Symfony sélectionne automatiquement Argon2id si disponible, sinon bcrypt. Les paramètres de coût sont réduits dans l'environnement de test pour accélérer l'exécution (cost: 4, time_cost: 3, memory_cost: 10).

Code PIN

Le PIN est haché avec password_hash() PHP (algorithme PASSWORD_DEFAULT, soit bcrypt avec coût adapté). Il est vérifié via password_verify(). Le PIN est requis pour révéler un secret marqué isSensitive = true.

8.2 Autorisation

Pare-feu stateless

```
# config/packages/security.yaml
firewalls:
```

```
    main:
      pattern: ^/api
      stateless: true
      custom_authenticators:
        - App\Security\Auth\ApiTokenAuthenticator
  access_control:
    - { path: ^/api/doc,      roles: PUBLIC_ACCESS }
    - { path: ^/api/auth,     roles: PUBLIC_ACCESS }
    - { path: ^/api/health,   roles: PUBLIC_ACCESS }
    - { path: ^/api/admin,    roles: ROLE_ADMIN }
    - { path: ^/api,          roles: ROLE_USER }
```

Voters Symfony

Voter	Attribut	Condition d'accès
VaultVoter	VIEW	Être membre du trousseau
	EDIT	Rôle OWNER ou EDITOR
	DELETE	Rôle OWNER uniquement
	MANAGE_MEMBERS	Rôle OWNER uniquement
ItemVoter	VIEW	Membre du vault ET canView = true (si ItemPermission existante)
	EDIT	Rôle EDITOR/OWNER ET canEdit = true
	DELETE	Rôle OWNER
	MANAGE_ATTACHMENTS	Rôle EDITOR/OWNER ET canManageAttachments = true
	REVEAL_SECRET	Membre ET canRevealSecret = true

8.3 Chiffrement des secrets

La classe **VaultSecretCipher** utilise les fonctions **sodium_crypto_secretbox / sodium_secretbox_open**. La clé de 32 octets est dérivée de **APP_SECRET** par **hash('sha256', APP_SECRET, true).**

Stocké en BDD : enc:v1:<base64(nonce[24 bytes] || ciphertext)>

Le préfixe **enc:v1:** permet la migration future : si un secret ne commence pas par ce préfixe, il est traité comme du texte en clair (compatibilité ascendante).

8.4 Limitation de débit (Rate Limiting)

Composant **symfony/rate-limiter** appliqué sur **POST /api/auth/login** :

- **Seuil :** 5 tentatives échouées par adresse IP sur une fenêtre glissante
- **Réponse :** HTTP 429 avec en-tête **Retry-After**
- **Test de non-régression : AuthFlowTest::testLoginIsRateLimitedAfterTooManyAttempts** vérifie que le 6e appel renvoie 429

8.5 CORS

La bundle **nelmio/cors-bundle** contrôle les origines autorisées via la variable d'environnement **CORS_ALLOW_ORIGIN**. En développement, seul **http://localhost:5173** est autorisé.

8.6 Invalidation de session

Lors de la déconnexion (POST /api/auth/logout) ou du changement de mot de passe, le champ **authTokenVersion** de l'utilisateur est incrémenté en base de données. Tout jeton émis antérieurement devient immédiatement invalide, même s'il n'a pas encore expiré. Ce mécanisme est validé par deux tests dédiés.

9. API et contrats d'échange

9.1 Conventions générales

- **Base URL :** http://localhost:8000/api
- **Format :** JSON (Content-Type: application/json)
- **Authentification :** en-tête Authorization: Bearer <token> sur toutes les routes protégées
- **Erreurs d'autorisation :** HTTP 403
- **Non authentifié :** HTTP 401

Format d'erreur standard

Toutes les réponses d'erreur sont homogénéisées par ApiRequestLifecycleSubscriber :

```
{
  "message": "Acces interdit.",
  "status": 403,
  "code": "forbidden",
  "timestamp": "2026-04-19T11:15:31+00:00",
  "path": "/api/vaults/12",
  "requestId": "c5db4f..."
}
```

Pour les erreurs de validation (HTTP 422), le champ errors est ajouté avec le détail par champ :

```
{
  "message": "Les données fournies sont invalides.",
  "errors": {
    "email": ["Cette adresse e-mail est déjà utilisée."],
    "password": ["Ce champ est requis."]
  }
}
```

Tracabilité

Chaque réponse API inclut l'en-tête **X-Request-Id**. En cas d'incident, cet identifiant permet de corréler la requête avec les logs serveur.

9.2 Tableau des routes

Authentification et profil

Méthode	Chemin	Auth	Description	Réponse succès
POST	/api/auth/register	Non	Créer un compte	201 {token, user}
POST	/api/auth/login	Non	Connexion	200 {token, user}
POST	/api/auth/logout	Oui	Déconnexion + invalidation jeton	200
GET	/api/auth/oauth/google/redirect	Non	Démarrer flux OAuth Google	302
GET	/api/auth/oauth/google/callback	Non	Callback OAuth Google	302
POST	/api/auth/oauth/google/confirm	Non	Confirmer création compte OAuth	200 {token, user}
GET	/api/me	Oui	Profil courant	200 {user}
PATCH	/api/me	Oui	Modifier profil/email/mot de passe	200 {token, user}
DELETE	/api/me	Oui	Supprimer son compte	200
POST	/api/me/verify-pin	Oui	Vérifier le PIN	200

Trousseaux

Méthode	Chemin	Auth	Description	Réponse succès
GET	/api/vaults	Oui	Lister les trousseaux accessibles (?q=)	200 {vaults[]}
POST	/api/vaults	Oui	Créer un trousseau	201 {vault}
GET	/api/vaults/{id}	Oui (membre)	Détail d'un trousseau	200 {vault}
PATCH	/api/vaults/{id}	Oui (OWNER/EDITOR)	Modifier nom/description	200 {vault}
DELETE	/api/vaults/{id}	Oui (OWNER)	Supprimer un trousseau	200
GET	/api/vaults/{id}/members	Oui (membre)	Lister les membres	200 {members[]}
POST	/api/vaults/{id}/members	Oui (OWNER)	Inviter un membre par email	201 {member, vaultType}
PATCH	/api/vaults/{id}/members/{mid}	Oui (OWNER)	Modifier le rôle d'un membre	200 {member}
DELETE	/api/vaults/{id}/members/{mid}	Oui (OWNER/soi-même)	Retirer/quitter un trousseau	200 {message, vaultType}
GET	/api/vaults/{id}/export	Oui (membre)	Exporter le trousseau (`?format=json\	csv`)

Éléments

Méthode	Chemin	Auth	Description	Réponse succès
GET	/api/vaults/{id}/items	Oui (membre)	Lister les éléments (?q=)	200 {items[]}
POST	/api/vaults/{id}/items	Oui (EDITOR/OWNER)	Créer un élément	201 {item}

GET	/api/items/{id}	Oui (VIEW)	Détail + secret déchiffré	200 {item}
POST	/api/items/{id}/unlock-secret	Oui (REVEAL_SECRET)	Révéler le secret (PIN si sensible)	200 {secret}
PATCH	/api/items/{id}	Oui (EDIT)	Modifier un élément	200 {item}
DELETE	/api/items/{id}	Oui (OWNER)	Supprimer un élément	200
POST	/api/items/{id}/attachments	Oui (MANAGE_ATTACHMENTS)	Uploader une pièce jointe	201 {attachment}
DELETE	/api/attachments/{id}	Oui (MANAGE_ATTACHMENTS)	Supprimer une pièce jointe	200
GET	/api/attachments/{id}/download	Oui	Télécharger une pièce jointe	200 (binaire)
GET	/api/attachments/{id}/preview	Oui	Prévisualiser une pièce jointe	200 (binaire)

Messagerie

Méthode	Chemin	Auth	Description
GET	/api/vaults/{id}/messages	Oui (membre)	Messages du chat de trousseau (?limit, ?since, ?before)
POST	/api/vaults/{id}/messages	Oui (membre)	Envoyer un message dans le chat de trousseau
GET	/api/messages/private/contacts	Oui	Lister les contacts de messagerie privée
GET	/api/messages/private/{userId}	Oui	Messages privés avec un utilisateur
POST	/api/messages/private/{userId}	Oui	Envoyer un message privé

Outils, import/export et statistiques

Méthode	Chemin	Auth	Description
POST	/api/tools/password/generate	Oui	Générer un mot de passe aléatoire (corps JSON avec options)
GET	/api/tools/export	Oui	Export global des données (`?format=json\
POST	/api/tools/import	Oui	Importer des données (multipart : champ file + format optionnel)
GET	/api/tools/stats	Oui	Statistiques utilisateur (vaults, items, membres)

Administration et système

Méthode	Chemin	Auth	Description
GET	/api/admin/users	ROLE_ADMIN	Lister tous les utilisateurs
PATCH	/api/admin/users/{id}	ROLE_ADMIN	Modifier un utilisateur (rôle, statut)
GET	/api/health	Non	Vérification santé du service
GET	/api/doc	Non	Documentation OpenAPI interactive (Swagger UI)

9.3 Exemples de contrats d'échange

POST /api/auth/register

Corps de requête :

```
{
  "email": "camille@example.com",
  "password": "motdepasse123",
  "firstname": "Camille",
  "lastname": "Durand"
}
```

Réponse 201 :

```
{
  "token": "eyJ....<signature>",
  "user": {
    "id": 1,
    "email": "camille@example.com",
    "firstname": "Camille",
    "lastname": "Durand",
    "hasLocalPassword": true,
    "hasPin": false,
    "isActive": true
  }
}
```

POST /api/vaults/{id}/items

Corps de requête :

```
{
  "name": "Netflix",
  "username": "camille@example.com",
  "secret": "p@ssw0rd!",
  "notes": "Compte familial",
  "isSensitive": false,
  "uris": [
    { "label": "Connexion", "uri": "https://www.netflix.com/login" }
  ],
  "customFields": [
    { "label": "Profil", "type": "text", "value": "Famille", "isSensitive": false }
  ]
}
```

Réponse 201 (le secret n'est **pas** renvoyé dans la liste, seulement dans GET /api/items/{id}) :

```
{
  "item": {
    "id": 15,
    "name": "Netflix",
    "username": "camille@example.com",
    "hasSecret": true,
    "secret": null,
    "isSensitive": false,
```

"notes": "Compte familial",
"uris": [{ "id": 3, "label": "Connexion", "uri": "https://www.netflix.com/login" }],
"customFields": [{ "id": 7, "label": "Profil", "type": "text", "value": "Famille", "isSensitive": false }],
"createdAt": "2026-04-19T10:30:00+00:00",
"updatedAt": "2026-04-19T10:30:00+00:00"
}
}

10. Fonctionnalités majeures implémentées

10.1 Trousseaux et collaboration

Un trousseau est créé automatiquement lors de l'inscription (trousseau personnel initial). Son type (PERSONAL / SHARED) évolue dynamiquement : il devient **SHARED** dès qu'un second membre est ajouté, et revient à **PERSONAL** si ce membre quitte le trousseau. Ce calcul est effectué à chaque opération de modification de la liste des membres et renvoyé dans les réponses.

Invariant du propriétaire : le propriétaire ne peut pas être retiré du trousseau ni quitter celui-ci tant qu'il en est propriétaire. Un test fonctionnel (VaultFlowTest) vérifie ce comportement.

10.2 Déverrouillage des secrets sensibles

Pour un élément marqué **isSensitive = true**, la révélation du secret nécessite une preuve d'authentification supplémentaire transmise dans **POST /api/items/{id}/unlock-secret**. Deux formes de preuve sont acceptées :

- Un PIN valide (si l'utilisateur a configuré un PIN)
- Le mot de passe courant de l'utilisateur

Sans preuve valide, la réponse est HTTP 403. Cette vérification est indépendante de la permission **canRevealSecret** de l'ItemPermission : les deux conditions doivent être satisfaites.

10.3 OAuth Google (PKCE)

Le flux OAuth 2.0 avec Google est implémenté avec la protection PKCE (Proof Key for Code Exchange) :

- **GET /api/auth/oauth/google/redirect** - génère un state aléatoire, le stocke en session, redirige vers **accounts.google.com/o/oauth2/v2/auth**
- Google redirige vers **/api/auth/oauth/google/callback?code=...&state=...**
- L'API échange le code contre un token Google, récupère le profil utilisateur
- Si l'adresse email existe déjà en base, le compte OAuth est lié au compte existant
- Sinon, une identité **google_oauth_pending_identity** est stockée en session et le frontend est redirigé vers **/oauth/callback** pour confirmation explicite
- **POST /api/auth/oauth/google/confirm** finalise la création du compte et renvoie un jeton Bearer

Seul Google est supporté. Les providers GitHub, Microsoft ou autres ne sont pas implémentés.

10.4 Messagerie temps réel

Deux types de messagerie coexistent :

- **Chat de trousseau** (sw_vault_message) : tous les membres d'un trousseau peuvent lire et écrire

- **Messages privés** (sw_private_message) : entre deux utilisateurs identifiés, avec marquage de lecture (readAt)

A chaque nouveau message, le backend publie un événement sur le hub Mercure. Le frontend React écoute ce canal via EventSource (SSE). L'abonnement Mercure est sécurisé par un JWT signé avec MERCURE_JWT_SECRET.

10.5 Import / Export

Deux routes d'export coexistent :

- **GET /api/vaults/{id}/export?format=json|csv** : exporte un trousseau spécifique (accès membre requis)
- **GET /api/tools/export?format=json|csv** : export global via le module Outils

L'import s'effectue via POST /api/tools/import avec un fichier multipart (champ file) et un paramètre format optionnel (json ou csv).

L'export produit un fichier **JSON** ou **CSV** contenant la liste des éléments avec leurs secrets **en clair** (déchiffrés au moment de l'export). L'import recrée les éléments et re-chiffre les secrets avec la clé courante du serveur.

Avertissement de sécurité : le fichier exporté contient des données sensibles en clair. L'utilisateur en est informé dans l'interface. Cette limite est inhérente à l'architecture (chiffrement côté serveur).

11. Tests et stratégie de validation

11.1 Stratégie globale

Le projet adopte une stratégie de test en trois couches :

Couche	Outil	Portée	Environnement
Tests fonctionnels d'intégration	PHPUnit 9.5	API complète (requête -> réponse -> base)	SQLite in-memory
Tests unitaires composants	Vitest 2.1	Composants React isolés	jsdom
Tests bout-en-bout	Cypress 13 / Playwright 1.59	Parcours utilisateurs réels	Application Docker complète

11.2 Tests backend (PHPUnit)

Organisation : apps/api/tests/Functional/

Classe	Nombre de cas	Domaines couverts
AuthFlowTest	9	Inscription, connexion, déconnexion, profil, suppression, OAuth, rate limiting, admin
VaultFlowTest	7	CRUD vault, type PERSONAL/SHARED, membres, rôles, isolation entre utilisateurs
ItemFlowTest	5+	CRUD item, URIs, champs perso, permissions VIEWER, secrets sensibles, pièces jointes
MessagingFlowTest	Encours de dev	Messagerie privée, chat de trousseau
SmokeApiTest	Encours de dev	Vérifications d'accessibilité des endpoints publics

Classe de base `ApiTestCase` :

- Réinitialise la base de données avant chaque test (SQLite in-memory)
- Fournit **createUser**(array $overrides) et **authenticate**(User) pour la mise en place des scénarios
- Les hashs de mot de passe utilisent un coût réduit (bcrypt cost 4) pour la rapidité

Exemple de scénario de test :

AuthFlowTest::testLoginIsRateLimitedAfterTooManyAttempts effectue 5 appels avec un mot de passe invalide sur la même IP, puis vérifie que le 6e retourne HTTP 429 avec l'en-tête **Retry-After**.

Exemple de scénario de test :

VaultFlowTest::testVaultTypeBecomesSharedWhenAnotherMemberJoinsAndReturnsToPersonalWhenTheyLeave valide l'invariant du type de trousseau sur un cycle complet ajout/suppression de membre.

Lancer les tests backend :

```
cd apps/api
php bin/phpunit --testdox
```

11.3 Tests frontend (Vitest + Playwright + Cypress)

- **Vitest :** tests unitaires de composants React avec **@testing-library/react** et **jsdom**. Lancé avec **npm run test**.
- **Cypress :** Cypress : socle principal des tests E2E fonctionnels, avec plusieurs scénarios métier (authentification, navigation, trousseaux, items, messagerie, profil, dashboard), appuyés par des helpers dans cypress/helpers/. Exécution avec npm run test:e2e. Lancé avec **npm run test:e2e.**
- **Playwright** : utilisé principalement pour la régression visuelle via la suite playwright/ui-snapshots.spec.js, qui génère des captures d'écran complètes des pages et modales clés. Exécution avec **npm run test:playwright**.

12. Déploiement et exécution

12.1 Prérequis

- Docker Engine >= 24
- Docker Compose v2 (docker compose)
- Git

12.2 Configuration initiale

```
# 1. Cloner le dépôt
git clone <url_du_depot> supwarden
cd supwarden

# 2. Créer les fichiers d'environnement
cp .env.example .env
cp .env.local.example .env.local

# 3. Renseigner les variables obligatoires dans .env
# APP_SECRET=<chaine_aleatoire_32_chars>
# POSTGRES_DB=supwarden
# POSTGRES_USER=supwarden
# POSTGRES_PASSWORD=<mot_de_passe_fort>
# MERCURE_JWT_SECRET=<chaine_aleatoire>
# GOOGLE_OAUTH_CLIENT_ID=<id_google_console>
# GOOGLE_OAUTH_CLIENT_SECRET=<secret_google_console>
```

12.3 Variables d'environnement critiques

Variable	Rôle	Exemple de valeur
APP_SECRET	Clé de signature des jetons Bearer et de dérivation de la clé de chiffrement	Chaîne aléatoire de 32+ caractères
DATABASE_URL	DSN PostgreSQL (construit automatiquement par Docker Compose)	postgresql://user:pass@db:5432/supwarden
MERCURE_JWT_SECRET	Clé partagée entre l'API et le hub Mercure pour signer les JWT SSE	Chaîne aléatoire de 32+ caractères
GOOGLE_OAUTH_CLIENT_ID	Identifiant de l'application Google (console.cloud.google.com)	123456789-abc.apps.googleusercontent.com
GOOGLE_OAUTH_CLIENT_SECRET	Secret OAuth Google	GOCSPX-...
VITE_API_URL	URL de l'API côté frontend (injecté au build Vite)	http://localhost:8000

12.4 Démarrage en développement

```
# Démarrer tous les services (construction d'images incluse)
docker compose -f .docker-compose.yml --env-file .env --env-file .env.local up -d --build

# Vérifier l'état des services
docker compose ps

# Accès aux services
# API + doc Swagger : http://localhost:8000/api/doc
# Frontend React :    http://localhost:5173
# Adminer (BDD) :     http://localhost:8080
# Mercure Hub :       http://localhost:3000
```

Le script **scripts/start-api.sh** exécuté au démarrage du conteneur **api** lance automatiquement :

- **composer install** (si le répertoire **vendor/** est vide)
- **php bin/console doctrine:migrations:migrate --no-interaction** (appliquer les migrations)
- **php -S 0.0.0.0:8000 -t public/** (serveur de développement PHP)

12.5 Démarrage en production

```
cp .env.prod.example .env.prod
# Renseigner toutes les variables dans .env.prod

docker compose -f .docker-compose.prod.yml --env-file .env.prod up -d --build
```

En mode production, le Dockerfile du frontend compile l'application React (npm run build) et la sert via **Nginx**. Le backend est servi via **php-fpm** ou un serveur web configuré en production.

12.6 Santé des services

Docker Compose utilise des health checks pour orchestrer le démarrage dans l'ordre :

Service	Check	Intervalle
db	pg_isready	10s
api	GET /api/health	15s, start_period 20s
web	GET /	20s, start_period 30s

Le service **web** dépend du service **api** (condition service_healthy) et **api** dépend de **db** (condition service_healthy). Mercure est démarré indépendamment (service_started).

12.7 Commandes de maintenance

```
# Accéder au shell du conteneur API
docker compose exec api sh

# Appliquer les migrations manuellement
docker compose exec api php bin/console doctrine:migrations:migrate

# Lancer les tests backend
docker compose exec api php bin/phpunit --testdox

# Voir les logs d'un service
docker compose logs -f api

# Arrêter et nettoyer
docker compose down --remove-orphans
```

13. Prise en main développeur

Cette section s'adresse à un développeur rejoignant le projet pour la première fois. Elle détaille l'installation complète, la configuration, les workflows quotidiens et les points d'entrée dans le code.

13.1 Prérequis

Outil	Version minimale	Vérification
Git	2.x	git --version
Docker Engine	24+	docker --version
Docker Compose	v2	docker compose version
Node.js	20+	node --version
PHP	8.1+	php --version

Node.js et PHP ne sont pas nécessaires si toutes les commandes sont exécutées dans les conteneurs Docker. Ils sont utiles pour l'auto-complétion des éditeurs ou le développement local sans conteneur.

13.2 Installation pas à pas

```
# 1. Cloner le dépôt
git clone <url_du_depot> supwarden && cd supwarden
```

```
# 2. Copier les fichiers d'environnement
cp .env.example .env
cp .env.local.example .env.local   # surcharges locales (non versionnées)

# 3. Éditer .env (voir 13.3 pour le détail de chaque variable)

# 4. Générer un APP_SECRET aléatoire
php -r "echo bin2hex(random_bytes(32));"

# 5. Construire et démarrer tous les services
docker compose -f .docker-compose.yml --env-file .env --env-file .env.local up -d --build

# 6. Vérifier que tous les services sont healthy
docker compose ps
```

L'ordre de démarrage est géré automatiquement par les depends_on et healthcheck :

1. **db** (PostgreSQL) démarre en premier et attend **pg_isready**
2. **mercure** démarre indépendamment
3. **api** démarre quand **db** est **healthy**, applique les **migrations**, puis démarre le serveur **PHP**
4. **web** démarre quand **api** est **healthy**

13.3 Variables d'environnement détaillées

Variables obligatoires pour démarrer

Variable	Description	Valeur dev typique
APP_ENV	Environnement Symfony	dev
APP_SECRET	Clé de signature des jetons et de dérivation de la clé de chiffrement. Ne jamais partager ni réutiliser entre environnements.	Chaîne hex 64 caractères
APP_PORT	Port exposé de l'API	8000
POSTGRES_DB	Nom de la base de données	supwarden
POSTGRES_USER	Utilisateur PostgreSQL	supwarden
POSTGRES_PASSWORD	Mot de passe PostgreSQL	Chaîne quelconque en dev
DB_PORT	Port exposé de PostgreSQL	5432
MERCURE_JWT_SECRET	Clé partagée API <-> hub Mercure. Doit être identique dans les sections api et mercure du Compose.	Chaîne aléatoire 32+ caractères
MERCURE_PUBLIC_URL	URL publique du hub (navigateur)	http://localhost:3000/.well-known/mercure
MERCURE_PORT	Port exposé de Mercure	3000
VITE_API_URL	URL de l'API vue par le navigateur, injectée au build Vite (pas à l'exécution)	http://localhost:8000
WEB_PORT	Port exposé du frontend	5173

Variables OAuth Google (optionnelles en développement)

Variable	Description
GOOGLE_OAUTH_CLIENT_ID	ID client depuis console.cloud.google.com
GOOGLE_OAUTH_CLIENT_SECRET	Secret OAuth Google
GOOGLE_OAUTH_REDIRECT_URI	URI de callback déclarée dans Google Console (ex. http://localhost:8000/api/auth/oauth/google/callback)
GOOGLE_OAUTH_FRONTEND_CALLBACK_URL	URL du frontend pour finaliser le flux (ex. http://localhost:5173/oauth/callback)

Sans ces variables, la connexion Google renvoie une erreur 500 mais toutes les autres fonctionnalités restent opérationnelles.

13.4 Services et ports

Service Docker	Port local	URL	Description
api	8000	http://localhost:8000	API Symfony + Swagger UI à /api/doc
web	5173	http://localhost:5173	Frontend React (Vite HMR)
db	5432	—	PostgreSQL 16
adminer	8080	http://localhost:8080	Interface web PostgreSQL
mercure	3000	http://localhost:3000	Hub SSE Mercure

13.5 Cartographie du code

Backend (apps/api/src/)

Dossier	Contenu
Controller/Auth/	Inscription, connexion, déconnexion, OAuth Google
Controller/User/	Profil (/api/me), vérification PIN
Controller/Vault/	CRUD trousseaux, membres, export par trousseau
Controller/Item/	CRUD éléments, pièces jointes
Controller/Messaging/	Chat de trousseau (/api/vaults/{id}/messages), messages privés (/api/messages/private/*)
Controller/Tools/	Générateur de mots de passe, import/export global, statistiques
Controller/Admin/	Administration des utilisateurs (ROLE_ADMIN)
Entity/	11 entités Doctrine (préfixe de table sw_)
Dto/	Objets d'entrée validés par Symfony Validator (RegisterInput, LoginInput, CreateVaultInput...)
Enum/	Constantes métier typées : VaultMemberRole, VaultType, ItemType
EventSubscriber/	ApiRequestLifecycleSubscriber — injection X-Request-Id, homogénéisation des erreurs
Security/Token/	BearerTokenManager — création et validation des jetons
Security/Auth/	ApiTokenAuthenticator, AuthRateLimiter
Security/Vault/	VaultVoter, ItemVoter
Service/	VaultSecretCipher, PasswordGeneratorService, VaultDataExportService, VaultDataImportService
Command/	EncryptLegacyItemSecretsCommand, SeedUserVaultsCommand, EnsurePersonalVaultsCommand

Frontend (apps/web/src/)

Dossier/Fichier	Contenu
context/AuthContext.jsx	Contexte global d'authentification (token, user, logout)
routes/AppRouter.jsx	Routeur SPA personnalisé (pushState, gardes de route)
pages/auth/	Pages de connexion, inscription, callback OAuth
pages/vaults/	Liste, détail, création de trousseaux
pages/items/	Détail et formulaire d'élément
pages/messages/	Interface de messagerie
pages/settings/	Page de profil utilisateur
services/api/authApi.js	Appels auth et profil
services/api/vaultApi.js	Appels trousseaux et membres
services/api/itemApi.js	Appels éléments et pièces jointes
services/api/messagingApi.js	Appels messagerie
services/api/advancedApi.js	Outils, export, import, stats
hooks/	Hooks React personnalisés (session, notifications, SSE Mercure)
layouts/	Structures de page partagées (AppLayout, AuthLayout...)
utils/	Fonctions utilitaires (formatage, helpers)
styles/	Variables CSS, charte visuelle, styles globaux

13.6 Flux de développement quotidien

```
# Démarrer les services (si arrêtés)
docker compose -f .docker-compose.yml --env-file .env --env-file .env.local up -d

# Logs en temps réel
docker compose logs -f api      # logs backend Symfony
docker compose logs -f web      # logs Vite dev server

# Entrer dans le conteneur API
docker compose exec api sh

# Entrer dans le conteneur web
docker compose exec web sh
```

Le HMR (Hot Module Replacement) de Vite fonctionne en mode Docker : les modifications de fichiers dans **apps/web/src/** sont appliquées instantanément dans le navigateur grâce au volume monté.

Pour le backend, les modifications PHP sont prises en compte immédiatement (le serveur de développement PHP relit les fichiers à chaque requête).

13.7 Migrations de base de données

```
# Après modification d'une entité Doctrine :
# 1. Générer le fichier de migration
docker compose exec api php bin/console doctrine:migrations:diff
```

```
# 2. Vérifier le SQL généré dans apps/api/migrations/

# 3. Appliquer la migration
docker compose exec api php bin/console doctrine:migrations:migrate --no-interaction

# Vérifier l'état des migrations
docker compose exec api php bin/console doctrine:migrations:status

# Promouvoir un utilisateur en administrateur
docker compose exec api php bin/console doctrine:query:sql \
  "UPDATE sw_app_user SET roles = '[\"ROLE_USER\",\"ROLE_ADMIN\"]' WHERE email = 'admin@example.com'"
```

13.8 Lancer les tests

```
# Tests backend (PHPUnit — SQLite in-memory, pas de BDD Docker requise)
docker compose exec api php bin/phpunit --testdox

# Tests backend avec filtre de classe
docker compose exec api php bin/phpunit --filter AuthFlowTest --testdox

# Tests unitaires frontend (Vitest)
docker compose exec web npm run test

# Tests E2E Cypress (application complète en marche requise)
docker compose exec web npm run test:e2e

# Tests E2E Playwright (application complète en marche requise)
docker compose exec web npm run test:playwright
```

13.9 Reconstruire les images Docker

Nécessaire après modification d'un **Dockerfile**, de **composer.json** ou de **package.json** :

```
# Reconstruire une image spécifique
docker compose build api
docker compose build web

# Reconstruire tout et redémarrer
docker compose -f .docker-compose.yml --env-file .env --env-file .env.local up -d --build

# Réinitialiser complètement (ATTENTION : supprime la base de données locale)
docker compose down -v
docker compose -f .docker-compose.yml --env-file .env --env-file .env.local up -d --build
```

13.10 Problèmes fréquents

Problème	Cause probable	Solution
Le conteneur api reste en starting	Migration échouée ou db pas encore prêt	docker compose logs api pour diagnostiquer
HTTP 401 sur une route protégée	Jeton expiré ou version invalidée	Se reconnecter pour obtenir un nouveau jeton

Erreur APP_SECRET is not set	Variable d'environnement manquante	Vérifier que .env est copié et rempli
Call to undefined function sodium_crypto_secretbox	Extension PHP sodium absente	Vérifier que le Dockerfile installe php8.1-sodium
Port 8000 ou 5173 déjà utilisé	Autre processus sur ce port	Modifier APP_PORT / WEB_PORT dans .env.local
Erreur CORS sur le frontend	CORS_ALLOW_ORIGIN incorrecte	Vérifier config/packages/nelmio_cors.yaml
Messages SSE non reçus	MERCURE_JWT_SECRET incohérent	La clé doit être identique côté api et mercure
doctrine:migrations:diff ne génère rien	Cache Doctrine périmé	docker compose exec api php bin/console cache:clear

14. Difficultés rencontrées et solutions

14.1 Invalidation de jetons sans état (stateless)

Problème : dans une API stateless, il n'existe pas de session serveur. Invalider un jeton après une déconnexion ou un changement de mot de passe est intrinsèquement difficile.

Solution retenue : introduction du champ `authTokenVersion` (entier) dans l'entité `User`. Lors de la création d'un jeton, la version courante est incluse dans le payload. A la validation, la version du jeton est comparée à la version en base ; une discordance invalide le jeton. Un incrément en base suffit à invalider toute la session sans liste noire.

Inconvénient : nécessite une lecture en base à chaque requête pour vérifier la version.

14.2 Chiffrement côté serveur et portabilité

Problème : stocker les secrets chiffrés tout en permettant un export utilisable impose de déchiffrer côté serveur lors de l'export, exposant les données.

Solution retenue : le format `enc:v1:` permet d'identifier les valeurs chiffrées. L'export déchiffre tous les secrets. L'interface utilisateur avertit explicitement l'utilisateur avant l'export. Une migration de version de chiffrement serait possible sans migration de base de données en changeant le préfixe.

14.3 Calcul dynamique du type de trousseau

Problème : le type de trousseau (`PERSONAL` / `SHARED`) dépend du nombre de membres. Le stocker en base créerait des incohérences si la mise à jour était oubliée.

Solution retenue : le type n'est **pas** stocké en base. Il est calculé à chaque opération affectant les membres et renvoyé dans la réponse. Les tests vérifient le cycle complet (ajout -> SHARED, suppression -> PERSONAL).

14.4 Routage SPA frontend sans React Router

Problème : React Router apporte des dépendances et une API parfois instable entre versions majeures. Le projet voulait contrôler précisément la logique de garde de route.

Solution retenue : implémentation d'un AppRouter personnalisé avec window.history.pushState, une liste de routes avec pattern-matching (:id -> paramètre dynamique), et des gardes de route (routes privées -> redirection si non authentifié, routes publiques -> redirection si authentifié).

Limite : plus verbeux et sans support natif des fonctionnalités avancées (data loading, transitions, etc.).

14.5 Environnement de test unifié

Problème : les tests backend nécessitent une base de données. Utiliser PostgreSQL en CI ralentit les builds et complexifie la configuration.

Solution retenue : SQLite in-memory pour les tests PHPUnit. L'environnement test (défini dans when@test: de security.yaml) utilise un coût de hachage minimal. Les repositories utilisent des Query Builders Doctrine compatibles SQLite. Cette approche échange la parité stricte prod/test contre la rapidité d'exécution.

15. Limites connues

Domaine	Limite	Impact
Chiffrement	Côté serveur uniquement, non bout-en-bout	Les secrets sont lisibles par l'opérateur du serveur ayant accès à APP_SECRET
OAuth	Google uniquement	Pas de connexion GitHub, Microsoft, Apple, etc.
Recherche	Filtre LIKE côté base de données	Pas de recherche plein texte (FTS), sensible à la casse selon la collation
Pagination	Non implémentée sur les listes	Un trousseau avec de nombreux éléments retourne tout en une seule réponse
Notifications	SSE uniquement (unidirectionnel)	Pas de push notifications navigateur, pas de notifications email
Mobile	Application web responsive mais non native	Pas d'application iOS/Android
2FA	Non implémenté	Pas de TOTP/HOTP/WebAuthn
Audit trail	Non implémenté	Impossible de savoir qui a modifié quoi et quand

16. Perspectives d'amélioration

16.1 Sécurité

- **Chiffrement bout-en-bout (E2E) :** dériver la clé de chiffrement depuis le mot de passe utilisateur (PBKDF2 ou Argon2) côté client (WebCrypto API). Le serveur ne verrait plus jamais les secrets en clair.
- **Journalisation des accès :** enregistrement horodaté de chaque lecture de secret (qui, quand, depuis quelle IP).

16.2 Fonctionnel

- **Pagination et tri :** curseur de pagination sur les endpoints de liste, tri par nom/date.
- **Recherche améliorée :** recherche plein texte avec PostgreSQL **tsvector**, ou intégration d'un moteur de recherche (Elasticsearch/Meilisearch).

- **Corbeille :** suppression logique des éléments avec restauration possible.

16.3 Infrastructure

- **Cache Redis :** mise en cache des données de session du Rate Limiter et des résultats de requêtes fréquentes.
- **Stockage objet :** externaliser les pièces jointes vers S3 ou MinIO plutôt que le système de fichiers local du conteneur.
- **Rotation de clé :** mécanisme de re-chiffrement des secrets lors d'une rotation de `APP_SECRET`.
- **CI/CD :** pipeline GitHub Actions pour lint, tests, build et déploiement automatique.

17. Annexes techniques

17.1 Configuration Docker Compose complète (développement)

```
# .docker-compose.yml (extrait commenté)
services:
  api:
    build:
      context: .
      dockerfile: infra/docker/api/Dockerfile   # PHP 8.1-cli-alpine
    environment:
      APP_ENV: ${APP_ENV}
      APP_SECRET: ${APP_SECRET}
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      MERCURE_URL: http://mercure/.well-known/mercure   # URL interne inter-conteneurs
      MERCURE_PUBLIC_URL: ${MERCURE_PUBLIC_URL}        # URL publique navigateur
    healthcheck:
      test: ["CMD-SHELL", "wget -q -O - http://127.0.0.1:8000/api/health >/dev/null || exit 1"]
      interval: 15s
    depends_on:
      db:
        condition: service_healthy   # Attend que PostgreSQL soit prêt

  web:
    build:
      args:
        VITE_API_URL: ${VITE_API_URL}   # Injecté au BUILD Vite (pas à l'exécution)
    command: npm install && npm run dev -- --host 0.0.0.0

  db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]

  mercure:
    image: dunglas/mercure   # Hub Caddy officiel
    environment:
      MERCURE_PUBLISHER_JWT_KEY: ${MERCURE_JWT_SECRET}
      MERCURE_SUBSCRIBER_JWT_KEY: ${MERCURE_JWT_SECRET}
```

17.2 Structure du jeton Bearer

Format : <payload_b64url>.<signature_b64url>

Décodage payload :

```
{
  "sub": 42,          // ID utilisateur
  "iat": 1713600000,  // Emis a (Unix timestamp)
  "exp": 1713686400,  // Expire a (iat + TOKEN_TTL)
  "version": 3        // authTokenVersion de l'utilisateur
}
```

Signature : Base64Url(HMAC-SHA256(payload_b64url, APP_SECRET))

17.3 Format de stockage des secrets chiffrés

Valeur en base de données :
enc:v1:ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789==

Décodage :
- Préfixe : "enc:v1:"
- Après décodage Base64 :
  - Octets 0..23 : nonce (24 octets, XSalsa20)
  - Octets 24..N : ciphertext (AEAD, Poly1305 MAC inclus)

Clé de déchiffrement : SHA256(APP_SECRET, raw=true) -> 32 octets
Algorithme : sodium_crypto_secretbox_open(ciphertext, nonce, key)

17.4 Entités Doctrine - champs techniques

Champ roles (sw_app_user)

Stocké en JSON en base. Valeurs possibles : **["ROLE_USER"]** (défaut) ou **["ROLE_USER", "ROLE_ADMIN"]**. Symfony Security lit ce tableau pour évaluer **ROLE_ADMIN** dans l'access control.

Triggers Doctrine (#[ORM\PrePersist] / #[ORM\PreUpdate])

L'entité **VaultItem** utilise des lifecycle callbacks pour mettre à jour **createdAt** à l'insertion et **updatedAt** à chaque modification. Ces champs sont de type **DateTimeImmutable**.

17.5 Documentation OpenAPI

La documentation interactive est disponible à l'URL **/api/doc** en développement. Elle est générée automatiquement par **nelmio/api-doc-bundle** à partir des attributs PHP 8 **#[OA\Get], #[OA\Post], #[OA\RequestBody],** etc. sur les contrôleurs.

Le schéma de sécurité déclaré est **BearerAuth** (HTTP Bearer). Les routes publiques ne portent pas l'attribut **#[OA\Security].**

17.6 Commandes utiles

```
# Générer un APP_SECRET aléatoire
php -r "echo bin2hex(random_bytes(32));"

# Créer une nouvelle migration après modification d'entité
docker compose exec api php bin/console doctrine:migrations:diff
```

Appliquer les migrations
docker compose exec api php bin/console doctrine:migrations:migrate

Vérifier l'état des migrations
docker compose exec api php bin/console doctrine:migrations:status

Linter le frontend
docker compose exec web npm run lint

Lancer les tests unitaires frontend
docker compose exec web npm run test

Lancer les tests E2E Cypress et Playwright
docker compose exec web npm run test:e2e
docker compose exec web npm run test:playwright